

Programmation Orientée Objet pour le Calcul Scientifique : Part II Deep Water

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- **Outils et Langage (TooL)**
- Gestion de la mémoire (Mem)
- Orienté objet (OO)
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 13 : la bibliothèque standard STL (introduction)

Qu'est-ce que c'est ? Tout ce que vous trouvez sous `std::` ...

STL : Standard Template Library

- une bibliothèque bas-niveau mise à disposition par votre compilateur
 - il existe autant de versions que de compilateurs
- un ensemble :
 - de conteneurs génériques (indépendant du type contenu)
 - d'algorithmes génériques (indépendant du conteneur)
 - d'objets de base bien pratiques (io, mémoire, exceptions, parallélisme...)

Le premier réflexe à avoir quand on a besoin de quelque chose : est-ce dans la STL ?

Nous focalisons dans cet item sur les **conteneurs** et les **algorithmes**

Item 13 : la bibliothèque standard STL (conteneurs)

Les premiers conteneurs : paire et n-uplet 1/2

- Le conteneur le plus simple est la **paire d'élément** : `std::pair<T,U>`
 - les deux objets sont de **type quelconque**
 - on peut la construire avec la fonction `std::make_pair(a,b)`
 - il faut inclure le header `<utility>`
 - les éléments s'accèdent au moyen des données membres `first` et `second`

```
#include <utility>
int main() {
    std::pair<char,int> pair = std::make_pair('a',1);
    std::cout << pair.first << " ";
    std::cout << pair.second;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les premiers conteneurs : paire et n-uplet 2/2

- Le **n-uplet** stocke un nombre quelconque d'élément : `std::tuple<T...>`
 - les n objets sont de **type quelconque**
 - on peut le construire avec la fonction `std::make_tuple(a,b,c,d)`
 - il faut inclure le header `<tuple>`
 - les éléments s'accèdent au moyen de la fonction `std::get<i>(tuple)`

```
#include <tuple>
int main() {
    std::tuple<char,int,float> tuple = std::make_tuple('a',1,10.f);
    std::cout << std::get<0>(tuple) << " " ;
    std::cout << std::get<1>(tuple) << " " ;
    std::cout << std::get<2>(tuple) << " " ;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

La chaîne de caractère, un premier tableau

- la chaîne de caractère : `std::string`

```
#include <string>

int main(){
    std::string str{"Hello World"};
    std::cout << str << std::endl;
    str = {"new value"};
    std::cout << str << std::endl;
    auto size = str.size(); // = 9
    std::cout << str[size-1]; // e
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les tableaux de la STL 1/3

- **tableau** de taille statique `std::array<T,size>`

```
#include <array>

int main() {
    std::array<int,3> int_array_of_size_3 {0,1,2};
    int_array_of_size_3[2] = 0;

    // Deduction guide
    std::array deduced_size_type_array {0,1,2,3}; // -> std::array<int,4>

    std::array<char, 2> char_array = { 'a', 'b' };
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les tableaux de la STL 2/3

- **tableau** de taille dynamique `std::vector<T>`

```
#include <vector>

int main() {
    std::vector<int> vec {0,1};
    vec.push_back(2);

    vec.clear();

    vec = {2,4,5};
    vec[0] += 2;
}
```

De nombreuses fonctionnalités : <https://en.cppreference.com/w/cpp/container/vector>

Item 13 : la bibliothèque standard STL (conteneurs)

Les tableaux de la STL 3/3

- **tableau** de taille dynamique `std::vector<T>` : création et allocation

```
int main() { // do not forget to #include <vector>
    std::vector<int> vec{0,1}; // vector with 2 elements
    std::cout << vec[0] << " " << vec[1] << std::endl;
    std::vector<int> vec2; // empty vector
    vec.resize(2); //allocate 2 elements
    std::cout << vec[0] << " " << vec[1] << std::endl;
    vec.push_back(2); // add a new element : re-allocation
    std::cout << vec[2] << std::endl;
    std::vector<int> vec3; // empty vector
    vec3.reserve(3); // allocate a maximum size. size = 0, capacity = 3
    vec3.push_back(2); // add a new element : no re-allocation
    vec.clear(); vec2.clear(); vec3.clear();
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les autres conteneurs séquences de la STL

La STL contient d'autres conteneurs séquences (éléments non triés)

- un tableau à double entrée : `std::deque<T>` (`push_back()` et `push_front()`)
- des listes chaînées
 - coût d'insertion d'un éléments très faible (contrairement à `vector`)
 - **pas d'accès indexé à un élément** (`operator[](int)` inexistant)
 - la liste `std::list<T>` : `push_back()/push_front()` , `pop_back()/pop_front()`
 - la file `std::queue<T>` : manipulation FIFO (First In First Out, `push()/pop()`)
 - la pile `std::stack<T>` : manipulation LIFO (Last In First Out, `push()/pop()`)

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs séquences de la STL

- `std::array<T,int>`
- `std::vector<T>`
- `std::deque<T>` , `std::list<T>` , `std::queue<T>` , `std::stack<T>`

Exo 13.1 : manipulation de conteneurs séquences

- créez un conteneur `array` et `vector` ,
- les remplir (à la construction ou via `resize` et `push_back` pour le vecteur)
- affichez la taille (éventuellement la capacité).
- affichez leurs valeurs via une unique fonction template.

Item 13 : la bibliothèque standard STL (conteneurs)

Les vues sur les conteneurs séquences (contigus)

- `std::string_view` C++17, vue constante sur un `std::string`

```
#include <string_view>

int main(){
    std::string str{"Hello World"};
    std::string_view str_view = str; // No copy
    // Can be handled like a std::string
    std::cout << str_view << std::endl;
    auto size = str_view.size(); // = 9
    std::cout << str_view[size-1]; // e
    // str_view[0] = '0'; KO, not mutable
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les vues sur les conteneurs séquences (contigus)

- `std::span<T>` C++20, vue sur un tableau

```
#include <span>

int main(){
    int static_array[3] = {1,2,3};
    std::span<int> span1(static_array);
    std::array<int, 3> std_array = {4,5,6};
    std::span<int> span2(std_array);
    std::vector<int> std_vector = {7,8,9};
    std::span<int> span3(std_vector);
    std::span<int,3> span4(static_array);
    std::span<int,3> span5(std_array);
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les vues sur les conteneurs séquences (contigus)

- `std::span<T>` C++20, vue sur un tableau
- ne possède pas les données
- permet de modifier les données
- ne permet pas de redimensionner le conteneur
- est invalidée si le conteneur est redimensionné ou supprimé
- peut avoir sa taille fixée statiquement

Item 13 : la bibliothèque standard STL (conteneurs)

Les vues sur les conteneurs séquences (contigus)

- permettent une sémantique de passage par copie dans des fonctions
- attention à la validité du conteneur au sein de la fonction

```
#include <vector>
#include <span>

void function(std::span<int> view) {...}

int main(){
    std::vector vec{1,2,3};
    function(vec);
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 1/7

- Les conteneurs associatifs trient leurs éléments à l'insertion
 - accès énuméré ou par une recherche ; pas d'accès direct par un index
- Les deux principaux conteneurs associatifs sont
 - l'ensemble ou set `std::set<T>`
 - la dictionnaire clé-valeur ou map `std::map<Key,T>`

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 2/7

- l'ensemble ou set `std::set<T>`
 - par défaut les éléments sont triés par un `operator<`
 - il est possible de donner une comparaison défini par l'utilisateur
 - `std::set<T, Compare>`
 - le set n'insère qu'une fois une même valeur, pas de doublon
 - voir `std::multiset` pour un set contenant des doublons
 - la recherche d'une valeur retourne un itérateur (syntaxe de pointeur)
 - les éléments du set ne sont pas modifiables, stockés en `const T`

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 3/7

- l'ensemble ou set `std::set<T>`

```
#include <set>

int main(){
    std::set<int> my_set {4,1,7,2}; // inserted element will be sorted

    my_set.insert(0); // inserted before 1
    auto value_ite = my_set.find(1); // returns an iterator

    if (value_ite != my_set.end()) {
        std::cout << *value_ite;
    }
    my_set.clear();
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 4/7

- l'ensemble ou set `std::set<T>`

```
#include <set>

int main(){
    std::set<double> my_set { 1.0, 2.0 };
    using my_iterator = std::set<double>::iterator;
    std::pair<my_iterator, bool> insert_ret = my_set.insert(2.0);
    if (! insert_ret.second) std::cout << "Insertion didn't occur" << std::endl;

    // Light syntax with structured bindings
    auto [iterator, is_inserted] = my_set.insert(4.0);
    if (! is_inserted) std::cout << "Insertion didn't occur";
    else std::cout << (*iterator);
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 5/7

- le dictionnaire clé-valeur ou map `std::map<Key, Value>`
 - stocke des couples clé/valeur sous la forme de
 - `std::pair<const Key, Value>`
 - l'insertion n'a lieu que si la clé n'existe pas : pas de clé en doublon
 - voir `std::multimap` pour des stockages avec clés en doublons
 - accès aux éléments via un itérateur pointant vers : `std::pair<const Key, Value>`
 - les clés ne sont pas modifiables

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 6/7

- le dictionnaire clé-valeur ou map `std::map<Key, Value>`

```
#include <map>

int main(){
    std::map<std::string, double> my_map { {"val1", 1.0}, {"val2", 2.0} };

    my_map.insert({"val3", 3.0});
    my_map.insert(std::make_pair("val4", 4.0));

    auto val_ite = my_map.find("val2");
    if (val_ite != my_map.end())
        std::cout << (*val_ite).second << std::endl;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL 7/7

- le dictionnaire clé-valeur ou map `std::map<Key, Value>`

```
#include <map>

int main(){
    std::map<std::string, double> my_map { {"val1", 1.0}, {"val2", 2.0} };
    using my_iterator = std::map<std::string, double>::iterator;
    std::pair<my_iterator, bool> insert_ret = my_map.insert({"val1", 3.0});
    if (! insert_ret.second) std::cout << "Insertion didn't occur" << std::endl;

    // Light syntax with structured bindings
    auto [iterator, is_inserted] = my_map.insert({"val2", 4.0});
    if (! is_inserted) std::cout << "Insertion didn't occur";
    else std::cout << (*iterator).second;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Les conteneurs associatifs de la STL

- Les deux principaux conteneurs associatifs sont
 - l'ensemble ou set `std::set<T>`
 - le dictionnaire clé-valeur ou map `std::map<Key, T>`

Exo 13.2 : manipulation de conteneurs associatifs

- créez un conteneur `set` et `map`,
- les remplir à la construction et via une méthode `insert`,
- recherchez une valeur et modifiez-la si possible.

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 1/10

- Un itérateur est un concept de la STL permettant le parcours d'une collection
- Tous les conteneurs STL donne accès à un itérateur
- `container.begin()` donne accès au premier élément du tableau
- `container.end()` **n'est pas le dernier élément du conteneur**
 - il **pointe après la fin du conteneur** (past-the-end)
 - si `iterator == container.end()` on est en dehors du conteneur
 - pour tester que l'on est encore dans un conteneur on fait donc
 - `if(iterator != container.end())`

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 2/10

- Un itérateur a une syntaxe de pointeur
 - Un itérateur **passe d'un élément à un autre** par l'opérateur `++`
 - Un itérateur **donne accès à l'élément courant** par l'opérateur `*`
- Les algorithmes de la STL utilisent les itérateurs et non directement les conteneurs
 - ils sont donc **indépendants du type de conteneurs**
- Ils existe plusieurs concepts d'itérateurs :
 - `InputIterator` , `OutputIterator` , `ForwardIterator`
 - `BidirectionalIterator`
 - `RandomAccessIterator`

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 3/10

- Il faut donc utiliser l'itérateur pour parcourir le conteneur
- la syntaxe est la même quel que soit le conteneur

```
std::vector<int> my_vector {1,2,3};  
for (auto iter = my_vector.begin(); iter != my_vector.end(); ++iter){  
    std::cout << *iter << std::endl;  
}
```

```
std::map<std::string,int> my_map { {"val1",1.0}, {"val2",2.0} };  
for (auto iter = my_map.begin(); iter != my_map.end(); ++iter){  
    std::cout << "key    : " << (*iter).first << std::endl;  
    std::cout << "value  : " << (*iter).second << std::endl;  
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 4/10

- Il faut donc utiliser l'itérateur pour parcourir le conteneur
- la syntaxe est la même quelle que soit le conteneur

Exo 13.3 : manipulation de conteneurs :

- reprenez vos conteneurs des exercices 13.1 et 13.2,
- parcourez-les avec un itérateur,
- créer une fonction template qui affiche les éléments
 - Pouvez-vous utiliser le même code pour tous les conteneurs ?

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 5/10

Une **nouvelle boucle** sur les conteneurs : range-based for loop

```
std::vector vec {0,1,2}; // deduced to be std::vector<int>
for (int val : vec) { // val is a copy of a vec element
    std::cout << val << std::endl;
}
```

Attention à la déclaration de `val` : si l'on souhaite modifier le conteneur : il faut un `int&`

```
std::vector vec {0,1,2}; // deduced to be std::vector<int>
for (int& val : vec) { // val is a ref toward a vec element
    val += 1;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Itérer sur un conteneur STL 6/10

Une **nouvelle boucle** sur les conteneurs : range-based for loop

```
std::map<std::string,int> my_map { {"val1",1.0}, {"val2",2.0} };  
for (std::pair<const std::string,int> val : my_map) { // val is a copy of a map element  
    std::cout << "key    : " << val.first << std::endl;  
    std::cout << "value  : " << val.second << std::endl;  
}
```

Attention à la déclaration de `val` : si l'on souhaite modifier la valeur (la clé est const) :
il faut un `std::pair<const std::string,int>&`

```
std::map<std::string,int> my_map { {"val1",1.0}, {"val2",2.0} };  
for (std::pair<const std::string,int>& val : my_map) { // val is a copy of a map element  
    val.second +=2;  
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Boucle sur un conteneur STL 7/10

Une nouvelle boucle sur les conteneurs : range-based for loop.

S'utilise souvent avec `auto`

```
std::vector vec {0,1,2}; // deduced to be std::vector<int>
for (auto val : vec) { // val is a copy of a vec element
    std::cout << val << std::endl;
}
```

Attention à la déclaration de `val` : si l'on souhaite modifier le tableau : il faut `auto&`

```
std::vector vec {0,1,2}; // deduced to be std::vector<int>
for (auto& val : vec) { // val is a ref toward a vec element
    val += 1;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Boucle sur un conteneur STL 8/10

Une nouvelle boucle sur les conteneurs : range-based for loop.

S'utilise souvent avec `auto`

```
std::map<std::string,int> my_map { {"val1",1.0},{"val2",2.0} };
for (auto val : my_map) { // val is a copy of a map element
    std::cout << "key   : " << val.first << std::endl;
    std::cout << "value : " << val.second << std::endl;
}
```

Attention à la déclaration de `val` : si l'on souhaite modifier la valeur : il faut `auto&`

```
std::map<std::string,int> my_map { {"val1",1.0},{"val2",2.0} };
for (auto& val : my_map) { // val is a ref toward a map element
    val.second +=2;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Boucle sur un conteneur STL 9/10

Une nouvelle boucle sur les conteneurs : range-based for loop.

Grâce au **structured bindings** on peut aussi mapper directement la clé et la valeur :

```
std::map<std::string,int> my_map { {"val1",1.0},{"val2",2.0} };
for (auto [key,val] : my_map) { // key and val map key and value of the map element
    std::cout << "key    : " << key << std::endl;
    std::cout << "value  : " << val << std::endl;
}
```

Attention à la déclaration de `val` : si l'on souhaite modifier la valeur : il faut un `auto&`

```
std::map<std::string,int> my_map { {"val1",1.0},{"val2",2.0} };
for (auto& [key,val] : my_map) { // val is a ref toward the map element value
    val +=2;
}
```

Item 13 : la bibliothèque standard STL (conteneurs)

Boucle sur un conteneur STL 10/10

Une nouvelle boucle sur les conteneurs : range-based for loop

- fonctionne pour pour **tous les conteneurs de la stl**
- plus largement peut fonctionner pour une structure utilisateur
 - dès lors qu'elle propose des méthodes `begin()` et `end()`
 - donc dès qu'elle possède un itérateur
- **Exo 13.4** : modifiez l'exercice 13.3 avec des range-for loop
- **Exo 13.5 (Bonus)** créez votre propre conteneur
 - parcourez le avec une range-for loop
 - pour vous aider, faites le embarquer un conteneur STL...

Item 14 : la bibliothèque standard 2 (algorithmes)

Les **algorithmes** 1/3, permettent d'appliquer des traitements à des conteneurs

- indépendamment du type de conteneur
- en utilisant les itérateurs des conteneurs

Les algorithmes de la STL sont classés en :

- non modifying sequence : `for_each` , `find` , `find_if` , `search` , `any` , `count` , `count_if` ...
- modifying sequence : `fill` , `transform` , `generate` , `copy` , `copy_if` , `remove` , `remove_if` ...
- et bien plus encore : tri, partitionnement, numérique, opérations sur les set, min/max, comparaison..

cf. <https://en.cppreference.com/w/cpp/header/algorithm>

Item 14 : la bibliothèque standard 2 (algorithmes)

Les algorithmes 2/3, quelques exemples; ne pas oublier `#include <algorithm>`

```
std::vector<int> vec {0,1,1,2};
int num_ones = std::count(vec.begin(),vec.end(),1);

auto iter_max= std::max_element(vec.begin(),vec.end());
std::cout << *iter_max << std::endl;

std::vector<int> vec_copy;
std::copy(vec.begin(),vec.end(),std::back_inserter(vec_copy));

std::fill(vec.begin(),vec.end(),0);

bool is_equal = std::equal(vec.begin(),vec.end(),vec_copy.begin());

auto iter = std::find(vec.begin(),vec.end(),0);
if (iter != vec.end()) { std::cout << "vec contains " << *iter;}
```

Item 14 : la bibliothèque standard 2 (algorithmes)

Les algorithmes 2/3, quelques exemples

Exo 14.1 Manipulation d'algorithmes

- Construire un tableau contenant les entiers de 1 à 100 grâce à l'algorithme `iota`.
Attention cet algorithme est dans le header `numeric` ;
- Calculer la somme des éléments de ce tableau grâce à l'algorithme `accumulate` (header `numeric` également).

Item 14 : la bibliothèque standard 2 (algorithmes)

Les algorithmes 3/3, quelques exemples avec traitement

```
struct Function{  
    void operator()(int i ) {std::cout << i << ' ' ;} // treatment function  
};  
  
int main(){  
    std::vector<int> vec {0,1,2,3};  
    std::for_each(vec.begin(),vec.end(),Function{});  
}
```

- De nombreux algorithmes appellent une fonction de traitement sur les éléments
- Deux méthodes pour implémenter cette fonction :
 - une classe 'fonctor' qui se comporte comme une fonction (cf. ci-dessus)
 - une fonction anonyme (λ -function) qui peut s'écrire "en ligne"

Item 14 : la bibliothèque standard 2 (algorithmes)

Les algorithmes

Exo 14.2 Manipulation d'algorithmes avec traitement

- Reprenez votre exercice 14.1
- Modifier l'appel à `accumulate` pour calculer le double de la somme de tous les éléments. Vous utiliserez un fonctor comme dans le transparent précédent (mais attention `accumulate` a besoin d'une opération binaire, avec comme signature `int op(int sum, int current_element)`).

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 1/9

Une syntaxe assez particulière mais très employée, introduite en C++11

- On définit les arguments `()` le corps `{}` et une zone de capture `[]`

```
auto lambda = []() {return "Hello lambda world";};  
std::cout << lambda();
```

- la zone `()` permet de prendre des arguments d'entrée/sortie'

```
auto j = 1;  
auto lambda1 = [](int i) {return i++;};  
std::cout << lambda1(j); // j not modified  
auto lambda2 = [](int& i) {return i++;};  
lambda2(j); // j modified
```

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 2/9

Une syntaxe assez particulière mais très employée, introduite en C++11

- On définit les arguments `()` le corps `{}` et une zone de capture `[]`

```
auto lambda = []() {return "Hello lambda world";};  
std::cout << lambda();
```

- la zone `[]` permet d'embarquer des variables de l'environnement

```
int i = 1;  
auto lambda = [i]() {return i;};  
std::cout << lambda();
```

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 3/9

Une syntaxe assez particulière mais très employée, introduite en C++11

- Le type de retour est déduit automatiquement, mais on peut le préciser

```
auto lambda = []() -> std::string {return "Hello lambda world";};  
std::cout << lambda();
```

- le type de la lambda est créé automatiquement par le compilateur
 - il contient la signature complète de la lambda (capture comprise)
 - il est unique

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 4/9

Une syntaxe assez particulière mais très employée, introduite en C++11

- le type de la lambda est créé automatiquement par le compilateur
 - il contient la signature complète de la lambda (capture comprise)
 - il est unique
- on ne peut donc pas copier une lambda dans une autre
 - même avec une signature identique

```
auto lambda = [](int i) {return i++;};  
lambda = [](int i) {return 2*i;}; // Error not known conversion
```

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 5/9

Une syntaxe assez particulière mais très employée, introduite en C++11

- on ne peut donc pas copier une lambda dans une autre
- mais on peut copier dans un objet de type `std::function`
 - défini avec la même signature

```
auto lambda = [](int i) {return i++;};  
std::function<int(int)> function = lambda; // header <functional>  
std::cout << function(1);
```

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 6/9

La zone de capture :

- il est possible de capturer une variable par référence `[&i]` ou par copie `[i]`

```
int i = 1;
auto lambda1 = [i]() { return i + 1; };
std::cout << lambda1();
std::cout << " i = " << std::endl; // 1
auto lambda2 = [&i]() { return ++i; };
lambda2();
std::cout << " i = " << std::endl; // 2
auto lambda3 = [i]() { return ++i; }(); // ERROR, increment read-only variable
```

- on peut capturer toutes les variables de l'environnement (non recommandé)
 - par copie `[=]`

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 7/9

- on peut capturer toutes les variables de l'environnement (non recommandé)
 - par copie [=]
 - par référence [&]

```
int a = 1;
bool b = true;
auto lambda = [=]() { auto i = 0; if (b) i += a; return i; };
lambda();
auto lambda2 = [&]() { a++; b = false; };
lambda2();
```

Exo 15.1 : Créez une λ -function qui retourne le double de l'entier pris en argument, ou pris par capture. Avec la capture, faite une version où l'argument capturé est modifié.

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 8/9

Pourquoi utiliser des λ ? A la place de foncteurs ou de pointeurs de fonction

- pour passer des fonctions en argument
- un bon exemple : les algorithmes STL

```
std::vector<int> vec {0,1,2,3};  
  
std::for_each(vec.begin(),vec.end(),[](auto& i){i++;});
```

Exo 15.2 : reprenez l'exercice 14.2 et remplacez le foncteur par une λ fonction. Faites deux versions : une avec une prédéclaration de la λ et une où vous écrivez directement la λ dans l'algorithme.

Item 15 : les fonctions anonymes ou λ -functions

Les λ -functions 9/9

Pourquoi utiliser des λ ? A la place de foncteurs ou de pointeurs de fonction

- pour passer des fonctions en argument
- un bon exemple : les algorithmes STL

```
std::vector<int> vec {0,1,2,3};  
  
std::for_each(vec.begin(),vec.end(),[](auto& i){i++;});
```

Exo 15.3 : à l'aide d'un algorithme et d'une λ , copier les éléments pairs d'un vecteur dans un autre vecteur.